LAPORAN TUGAS PRAKTIKUM STRUCTUR DATA

PEKAN 9

DFS DAN BFS

Disusun Oleh :

UMAR ABDUL AZIZ

NIM 2511532012

DOSEN PENGAMPU Wahyudi Dr.S.T.M.T

DEPARTMENT INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG 14 MEI 2026

KATA PENGHANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kedua dengan judul "Double Linked List". Melalui penyusunan laporan tugas ini, penulis berharap dapat memberikan gambaran mengenai arraylist dengan java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 14 Mei 2026

Penulis

DAFTAR ISI dsadwadsad

BAB I
PENDAHULUAN

1.1 Latar Belakang

Doubly Linked List merupakan salah satu struktur data linear yang memiliki kemampuan untuk menghubungkan setiap node ke node berikutnya dan node sebelumnya. Struktur data ini sangat berguna dalam pengelolaan data yang membutuhkan proses penelusuran dua arah, baik dari depan ke belakang maupun dari belakang ke depan. Dengan adanya pointer next dan prev, proses manipulasi data seperti penambahan, penghapusan, dan pencarian data menjadi lebih fleksibel dibandingkan single linked list.

Pada praktikum ini dibuat sebuah program sederhana pengelolaan playlist musik menggunakan konsep List. Setiap lagu direpresentasikan sebagai sebuah node yang menyimpan informasi judul lagu, penyanyi, serta pointer ke lagu sebelumnya dan berikutnya. Program dirancang agar pengguna dapat menambahkan lagu ke playlist, menghapus lagu pertama, menampilkan daftar lagu secara maju maupun mundur, serta mencari lagu berdasarkan judul. Melalui praktikum ini diharapkan mahasiswa dapat memahami implementasi struktur data Doubly Linked List dalam bahasa pemrograman Java serta memahami cara kerja pointer next dan prev dalam pengelolaan data dinamis.

1.2 Tujuan Praktikum

- Memahami konsep dan penerapan struktur data double BFS dan DFS dalam bahasa pemrograman Java
- Mengimplementasikan operasi dasar BFS dan DFS seperti menambah data, menghapus data, menampilkan data maju dan mundur, serta mencari data.
- Melatih kemampuan dalam membuat program pengelolaan playlist musik menggunakan struktur data Doubly Linked List.

1.3 Persyaratan Praktikum

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.
- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Enviroment (IDE) yang direkomendasikan.
- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

BAB II
TUGAS PRAKTIKUM

2.1 Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph.

2.2 Kode Program

- **Coding**

```
package TugasPekan9_2511532012;
import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.geom.*;
import java.util.*;
import java.util.List;

public class PetaKampus_2311532012 extends JFrame {

    private Map<String, List<String>> graph_2012;
    private Map<String, Point> posisiNode_2012;
    private Map<String, Color> warnaNode_2012;
    private JComboBox<String> startCombo_2012;
    private JComboBox<String> goalCombo_2012;
    private JTextArea hasilArea_2012;
    private JLabel jumlahNodeLabel_2012;
    private GraphPanel graphPanel_2012;
    private List<String> jalurHasil_2012;
    private List<String> nodeDikunjungi_2012;

    private final String[] NODES_2012 = {
        "FTI", "FK", "FKG", "FH", "FT",
        "LABKOM 1", "LABKOM 2", "LABKOM 3",
        "PERPUSTAKAAN", "MESJID"
    };

    public PetaKampus_2311532012() {
        setTitle("Pencarian Jalur Kampus - BFS & DFS (2311532012)");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1300, 800);
        setLocationRelativeTo(null);

        initGraph_2012();
        initNodePositions_2012();
        initGUI_2012();

        setVisible(true);
```

```
    }

    private void initGraph_2012() {
        graph_2012 = new HashMap<>();

        for (String node : NODES_2012) {
            graph_2012.put(node, new ArrayList<>());
        }

        graph_2012.get("FTI").addAll(Arrays.asList("FK", "FKG", "FH"));
        graph_2012.get("FK").addAll(Arrays.asList("FTI", "FKG",
"PERPUSTAKAAN"));
        graph_2012.get("FKG").addAll(Arrays.asList("FTI", "FK", "FT"));
        graph_2012.get("FH").addAll(Arrays.asList("FTI", "FT", "MESJID"));
        graph_2012.get("FT").addAll(Arrays.asList("FKG", "FH", "LABKOM 1"));
        graph_2012.get("LABKOM 1").addAll(Arrays.asList("FT", "LABKOM 2"));
        graph_2012.get("LABKOM 2").addAll(Arrays.asList("LABKOM 1", "LABKOM
3"));
        graph_2012.get("LABKOM 3").addAll(Arrays.asList("LABKOM 2",
"PERPUSTAKAAN"));
        graph_2012.get("PERPUSTAKAAN").addAll(Arrays.asList("FK", "LABKOM
3", "MESJID"));
        graph_2012.get("MESJID").addAll(Arrays.asList("FH",
"PERPUSTAKAAN"));

        for (String node : graph_2012.keySet()) {
            List<String> neighbors = new ArrayList<>(new
LinkedHashSet<>(graph_2012.get(node)));
            graph_2012.put(node, neighbors);
        }
    }

    private void initNodePositions_2012() {
        posisiNode_2012 = new HashMap<>();

        posisiNode_2012.put("FK", new Point(80, 180));
        posisiNode_2012.put("FTI", new Point(80, 320));
        posisiNode_2012.put("PERPUSTAKAAN", new Point(80, 480));

        posisiNode_2012.put("FKG", new Point(280, 220));
        posisiNode_2012.put("FH", new Point(280, 380));
        posisiNode_2012.put("MESJID", new Point(280, 540));

        posisiNode_2012.put("FT", new Point(600, 300));
        posisiNode_2012.put("LABKOM 1", new Point(800, 240));
        posisiNode_2012.put("LABKOM 2", new Point(800, 140));
        posisiNode_2012.put("LABKOM 3", new Point(800, 40));

        warnaNode_2012 = new HashMap<>();
        for (String node : NODES_2012) {
            warnaNode_2012.put(node, new Color(70, 130, 200));
        }
    }

    private void initGUI_2012() {
```

```
        setLayout(new BorderLayout(10, 10));
        getContentPane().setBackground(new Color(240, 248, 255));

        JPanel northPanel = new JPanel();
        northPanel.setBackground(new Color(25, 118, 210));
        northPanel.setBorder(BorderFactory.createEmptyBorder(15, 10, 15,
10));

        JLabel titleLabel = new JLabel("PENCARIAN JALUR KAMPUS");
        titleLabel.setFont(new Font("Segoe UI", Font.BOLD, 22));
        titleLabel.setForeground(Color.WHITE);
        northPanel.add(titleLabel);

        JLabel subTitleLabel = new JLabel("BFS vs DFS - 2311532012");
        subTitleLabel.setFont(new Font("Segoe UI", Font.PLAIN, 14));
        subTitleLabel.setForeground(new Color(255, 255, 200));
        northPanel.add(subTitleLabel);

        add(northPanel, BorderLayout.NORTH);

        JPanel controlPanel = new JPanel(new GridBagLayout());
        controlPanel.setBorder(BorderFactory.createTitledBorder(
            BorderFactory.createLineBorder(new Color(25, 118, 210), 2),
            "Pengaturan Pencarian",
            TitledBorder.LEFT, TitledBorder.TOP,
            new Font("Segoe UI", Font.BOLD, 14),
            new Color(25, 118, 210)
        ));
        controlPanel.setBackground(new Color(255, 255, 255));

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(10, 15, 10, 15);

        gbc.gridx = 0; gbc.gridy = 0;
        JLabel startLabel = new JLabel("Lokasi Awal:");
        startLabel.setFont(new Font("Segoe UI", Font.BOLD, 14));
        controlPanel.add(startLabel, gbc);

        gbc.gridx = 1;
        startCombo_2012 = new JComboBox<>(NODES_2012);
        startCombo_2012.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        startCombo_2012.setPreferredSize(new Dimension(180, 30));
        controlPanel.add(startCombo_2012, gbc);

        gbc.gridx = 2;
        JLabel goalLabel = new JLabel("Lokasi Tujuan:");
        goalLabel.setFont(new Font("Segoe UI", Font.BOLD, 14));
        controlPanel.add(goalLabel, gbc);

        gbc.gridx = 3;
        goalCombo_2012 = new JComboBox<>(NODES_2012);
        goalCombo_2012.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        goalCombo_2012.setPreferredSize(new Dimension(180, 30));
        goalCombo_2012.setSelectedIndex(NODES_2012.length - 1);
        controlPanel.add(goalCombo_2012, gbc);
```

```
        gbc.gridx = 4;
        JButton bfsBtn = new JButton("BFS");
        bfsBtn.setFont(new Font("Segoe UI", Font.BOLD, 14));
        bfsBtn.setBackground(new Color(76, 175, 80));
        bfsBtn.setForeground(Color.WHITE);
        bfsBtn.setFocusPainted(false);
        bfsBtn.setPreferredSize(new Dimension(100, 40));
        bfsBtn.addActionListener(e -> runBFS_2012());
        controlPanel.add(bfsBtn, gbc);

        gbc.gridx = 5;
        JButton dfsBtn = new JButton("DFS");
        dfsBtn.setFont(new Font("Segoe UI", Font.BOLD, 14));
        dfsBtn.setBackground(new Color(33, 150, 243));
        dfsBtn.setForeground(Color.WHITE);
        dfsBtn.setFocusPainted(false);
        dfsBtn.setPreferredSize(new Dimension(100, 40));
        dfsBtn.addActionListener(e -> runDFS_2012());
        controlPanel.add(dfsBtn, gbc);

        gbc.gridx = 6;
        JButton resetBtn = new JButton("RESET");
        resetBtn.setFont(new Font("Segoe UI", Font.BOLD, 14));
        resetBtn.setBackground(new Color(244, 67, 54));
        resetBtn.setForeground(Color.WHITE);
        resetBtn.setFocusPainted(false);
        resetBtn.setPreferredSize(new Dimension(100, 40));
        resetBtn.addActionListener(e -> resetGraph_2012());
        controlPanel.add(resetBtn, gbc);

        add(controlPanel, BorderLayout.NORTH);

        graphPanel_2012 = new GraphPanel();
        graphPanel_2012.setBorder(BorderFactory.createLineBorder(new
Color(200, 200, 200), 2));

        JScrollPane scrollGraphPane = new JScrollPane(graphPanel_2012);
        scrollGraphPane.setPreferredSize(new Dimension(1000, 480));

scrollGraphPane.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBA
R_AS_NEEDED);

scrollGraphPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_AS
_NEEDED);
        add(scrollGraphPane, BorderLayout.CENTER);

        JPanel resultPanel = new JPanel(new BorderLayout(10, 10));
        resultPanel.setBorder(BorderFactory.createTitledBorder(
            BorderFactory.createLineBorder(new Color(25, 118, 210), 2),
            "Hasil Pencarian",
            TitledBorder.LEFT, TitledBorder.TOP,
            new Font("Segoe UI", Font.BOLD, 14),
            new Color(25, 118, 210)
        ));
```

```
        resultPanel.setBackground(new Color(255, 255, 255));
        resultPanel.setPreferredSize(new Dimension(1300, 180));

        hasilArea_2012 = new JTextArea();
        hasilArea_2012.setFont(new Font("Consolas", Font.PLAIN, 13));
        hasilArea_2012.setEditable(false);
        hasilArea_2012.setBackground(new Color(250, 250, 250));
        hasilArea_2012.setBorder(BorderFactory.createEmptyBorder(10, 10, 10,
10));

        JScrollPane scrollPane = new JScrollPane(hasilArea_2012);
        scrollPane.setBorder(BorderFactory.createLineBorder(new Color(200,
200, 200)));
        resultPanel.add(scrollPane, BorderLayout.CENTER);

        JPanel infoPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        infoPanel.setBackground(new Color(255, 255, 255));
        jumlahNodeLabel_2012 = new JLabel("Jumlah Node Dikunjungi: 0");
        jumlahNodeLabel_2012.setFont(new Font("Segoe UI", Font.BOLD, 13));
        jumlahNodeLabel_2012.setForeground(new Color(244, 67, 54));
        infoPanel.add(jumlahNodeLabel_2012);
        resultPanel.add(infoPanel, BorderLayout.SOUTH);

        add(resultPanel, BorderLayout.SOUTH);
    }

    private List<String> bfs_2012(String start, String goal, List<String>
visitedOrder) {
        Queue<List<String>> queue_2012 = new LinkedList<>();
        Set<String> visited_2012 = new LinkedHashSet<>();

        List<String> startPath_2012 = new ArrayList<>();
        startPath_2012.add(start);
        queue_2012.add(startPath_2012);
        visited_2012.add(start);
        visitedOrder.add(start);

        while (!queue_2012.isEmpty()) {
            List<String> path_2012 = queue_2012.poll();
            String node_2012 = path_2012.get(path_2012.size() - 1);

            if (node_2012.equals(goal)) {
                return path_2012;
            }

            List<String> neighbors_2012 = graph_2012.get(node_2012);
            if (neighbors_2012 != null) {
                for (String neighbor_2012 : neighbors_2012) {
                    if (!visited_2012.contains(neighbor_2012)) {
                        visited_2012.add(neighbor_2012);
                        visitedOrder.add(neighbor_2012);
                        List<String> newPath_2012 = new
ArrayList<>(path_2012);
                        newPath_2012.add(neighbor_2012);
                        queue_2012.add(newPath_2012);
```

```
                    }
                }
            }
        }
        return null;
    }

    private List<String> dfs_2012(String start, String goal, List<String>
visitedOrder) {
        Stack<List<String>> stack_2012 = new Stack<>();
        Set<String> visited_2012 = new LinkedHashSet<>();

        List<String> startPath_2012 = new ArrayList<>();
        startPath_2012.add(start);
        stack_2012.push(startPath_2012);
        visited_2012.add(start);
        visitedOrder.add(start);

        while (!stack_2012.isEmpty()) {
            List<String> path_2012 = stack_2012.pop();
            String node_2012 = path_2012.get(path_2012.size() - 1);

            if (node_2012.equals(goal)) {
                return path_2012;
            }

            List<String> neighbors_2012 = graph_2012.get(node_2012);
            if (neighbors_2012 != null) {
                for (int i = neighbors_2012.size() - 1; i >= 0; i--) {
                    String neighbor_2012 = neighbors_2012.get(i);
                    if (!visited_2012.contains(neighbor_2012)) {
                        visited_2012.add(neighbor_2012);
                        visitedOrder.add(neighbor_2012);
                        List<String> newPath_2012 = new
ArrayList<>(path_2012);
                        newPath_2012.add(neighbor_2012);
                        stack_2012.push(newPath_2012);
                    }
                }
            }
        }
        return null;
    }

    private void runBFS_2012() {
        String start_2012 = (String) startCombo_2012.getSelectedItem();
        String goal_2012 = (String) goalCombo_2012.getSelectedItem();
        List<String> visitedOrder_2012 = new ArrayList<>();

        long startTime_2012 = System.nanoTime();
        List<String> path_2012 = bfs_2012(start_2012, goal_2012,
visitedOrder_2012);
        long endTime_2012 = System.nanoTime();
        double duration_2012 = (endTime_2012 - startTime_2012) /
1_000_000.0;
```

```
        displayPath_2012(path_2012, visitedOrder_2012, "BFS",
duration_2012);

        jalurHasil_2012 = path_2012;
        nodeDikunjungi_2012 = visitedOrder_2012;
        updateNodeColors_2012(visitedOrder_2012, path_2012);
        graphPanel_2012.repaint();
    }

    private void runDFS_2012() {
        String start_2012 = (String) startCombo_2012.getSelectedItem();
        String goal_2012 = (String) goalCombo_2012.getSelectedItem();
        List<String> visitedOrder_2012 = new ArrayList<>();

        long startTime_2012 = System.nanoTime();
        List<String> path_2012 = dfs_2012(start_2012, goal_2012,
visitedOrder_2012);
        long endTime_2012 = System.nanoTime();
        double duration_2012 = (endTime_2012 - startTime_2012) /
1_000_000.0;

        displayPath_2012(path_2012, visitedOrder_2012, "DFS",
duration_2012);

        jalurHasil_2012 = path_2012;
        nodeDikunjungi_2012 = visitedOrder_2012;
        updateNodeColors_2012(visitedOrder_2012, path_2012);
        graphPanel_2012.repaint();
    }

    private void displayPath_2012(List<String> path, List<String> visited,
String algorithm, double duration) {
        StringBuilder sb_2012 = new StringBuilder();

        sb_2012.append("=".repeat(70)).append("\n");
        sb_2012.append(String.format("ALGORITMA: %s\n", algorithm));
        sb_2012.append("=".repeat(70)).append("\n\n");

        sb_2012.append("START:
").append(startCombo_2012.getSelectedItem()).append("\n");
        sb_2012.append("GOAL:
").append(goalCombo_2012.getSelectedItem()).append("\n\n");

        if (path != null && !path.isEmpty()) {
            sb_2012.append("JALUR YANG DITEMUKAN:\n");
            sb_2012.append("   ");
            for (int i = 0; i < path.size(); i++) {
                sb_2012.append(path.get(i));
                if (i < path.size() - 1) sb_2012.append(" -> ");
            }
            sb_2012.append("\n\n");
            sb_2012.append("Panjang Jalur: ").append(path.size() -
1).append(" langkah\n\n");
        } else {
            sb_2012.append("JALUR TIDAK DITEMUKAN!\n\n");
```

```
            }

            sb_2012.append("URUTAN NODE YANG DIKUNJUNGI:\n");
            sb_2012.append("   ");
            for (int i = 0; i < visited.size(); i++) {
                sb_2012.append(visited.get(i));
                if (i < visited.size() - 1) sb_2012.append(" -> ");
                if ((i + 1) % 5 == 0) sb_2012.append("\n   ");
            }
            sb_2012.append("\n\n");

            sb_2012.append("STATISTIK:\n");
            sb_2012.append("   Jumlah Node Dikunjungi:
").append(visited.size()).append("\n");
            sb_2012.append("   Waktu Eksekusi: ").append(String.format("%.3f",
duration)).append(" ms\n");

            sb_2012.append("\n").append("=".repeat(70));

            hasilArea_2012.setText(sb_2012.toString());
            jumlahNodeLabel_2012.setText("Jumlah Node Dikunjungi: " +
visited.size());
        }

        private void updateNodeColors_2012(List<String> visited, List<String>
path) {
            for (String node : NODES_2012) {
                warnaNode_2012.put(node, new Color(70, 130, 200));
            }

            if (visited != null) {
                for (String node : visited) {
                    warnaNode_2012.put(node, new Color(255, 193, 7));
                }
            }

            if (path != null) {
                for (String node : path) {
                    warnaNode_2012.put(node, new Color(76, 175, 80));
                }
            }

            String start = (String) startCombo_2012.getSelectedItem();
            String goal = (String) goalCombo_2012.getSelectedItem();
            if (start != null) warnaNode_2012.put(start, new Color(25, 118,
210));
            if (goal != null) warnaNode_2012.put(goal, new Color(244, 67, 54));
        }

        private void resetGraph_2012() {
            jalurHasil_2012 = null;
            nodeDikunjungi_2012 = null;

            for (String node : NODES_2012) {
                warnaNode_2012.put(node, new Color(70, 130, 200));
```

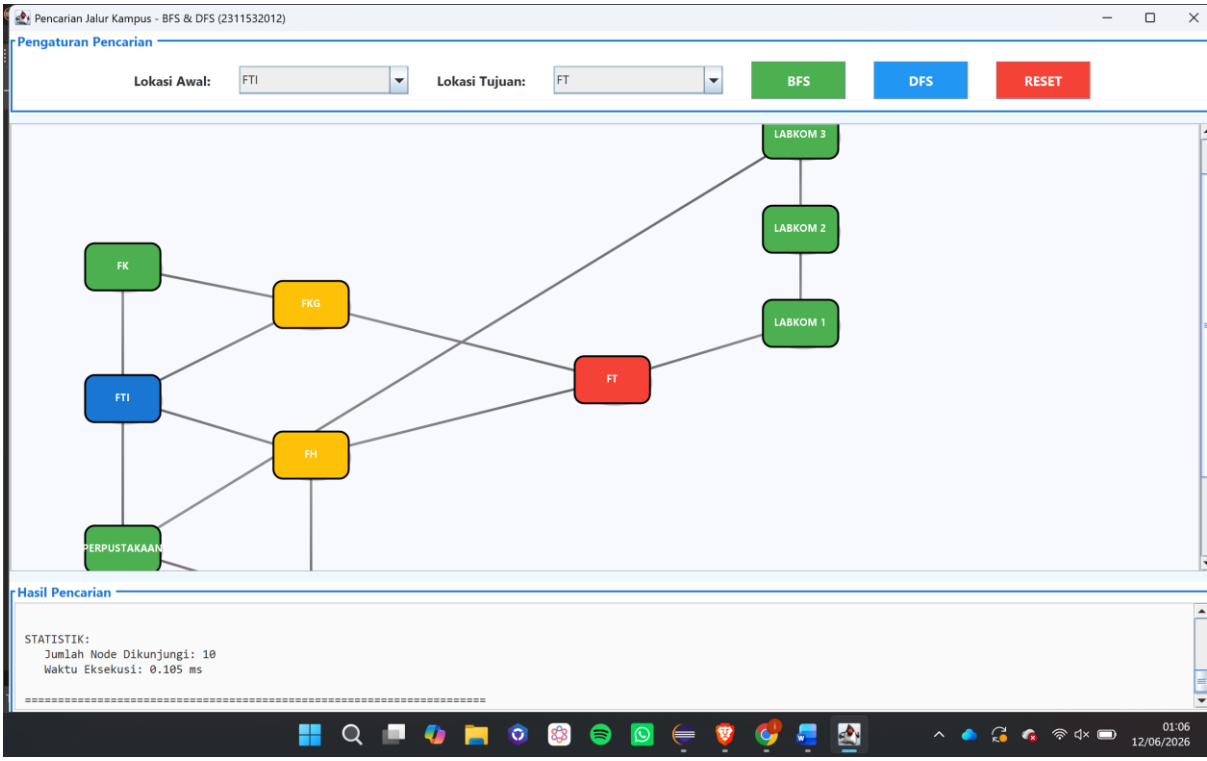
```
        }

        hasilArea_2012.setText("");
        jumlahNodeLabel_2012.setText("Jumlah Node Dikunjungi: 0");
        graphPanel_2012.repaint();
    }

    class GraphPanel extends JPanel {
        public GraphPanel() {
            setBackground(new Color(248, 248, 255));
            setPreferredSize(new Dimension(1100, 650));
        }

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2d_2012 = (Graphics2D) g;
            g2d_2012.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            g2d_2012.setStroke(new BasicStroke(2.5f));
            for (String node : graph_2012.keySet()) {
                Point from_2012 = posisiNode_2012.get(node);
                if (from_2012 == null) continue;

                for (String neighbor : graph_2012.get(node)) {
                    Point to_2012 = posisiNode_2012.get(neighbor);
                    if (to_2012 == null) continue;

                    GradientPaint gp_2012 = new GradientPaint(
                        from_2012.x + 40, from_2012.y + 25, new Color(100,
100, 100),
                        to_2012.x + 40, to_2012.y + 25, new Color(150, 150,
150)
                    );
                    g2d_2012.setPaint(gp_2012);
                    g2d_2012.drawLine(from_2012.x + 40, from_2012.y + 25,
to_2012.x + 40, to_2012.y + 25);

                    drawArrow(g2d_2012, from_2012.x + 40, from_2012.y + 25,
to_2012.x + 40, to_2012.y + 25);
                }
            }

            for (String node : NODES_2012) {
                Point p_2012 = posisiNode_2012.get(node);
                if (p_2012 == null) continue;

                Color nodeColor_2012 = warnaNode_2012.get(node);

                g2d_2012.setColor(new Color(0, 0, 0, 50));
                g2d_2012.fillOval(p_2012.x + 3, p_2012.y + 3, 80, 50);

                g2d_2012.setColor(nodeColor_2012);
                g2d_2012.fillRoundRect(p_2012.x, p_2012.y, 80, 50, 20, 20);
```

```
                g2d_2012.setColor(Color.BLACK);
                g2d_2012.setStroke(new BasicStroke(2));
                g2d_2012.drawRoundRect(p_2012.x, p_2012.y, 80, 50, 20, 20);

                g2d_2012.setFont(new Font("Segoe UI", Font.BOLD, 11));
                FontMetrics fm_2012 = g2d_2012.getFontMetrics();
                String displayText_2012 = node;
                int textWidth_2012 = fm_2012.stringWidth(displayText_2012);
                g2d_2012.setColor(Color.WHITE);
                g2d_2012.drawString(displayText_2012, p_2012.x + 40 -
textWidth_2012 / 2, p_2012.y + 28);
            }

            drawLegend(g2d_2012);
        }

        private void drawArrow(Graphics2D g2d, int x1, int y1, int x2, int
y2) {
            int arrowSize_2012 = 6;
            double angle_2012 = Math.atan2(y2 - y1, x2 - x1);
            int arrowX_2012 = (int) (x2 - 15 * Math.cos(angle_2012));
            int arrowY_2012 = (int) (y2 - 15 * Math.sin(angle_2012));

            int xPoints_2012[] = {
                arrowX_2012,
                (int) (arrowX_2012 - arrowSize_2012 * Math.cos(angle_2012 -
Math.PI / 6)),
                (int) (arrowX_2012 - arrowSize_2012 * Math.cos(angle_2012 +
Math.PI / 6))
            };
            int yPoints_2012[] = {
                arrowY_2012,
                (int) (arrowY_2012 - arrowSize_2012 * Math.sin(angle_2012 -
Math.PI / 6)),
                (int) (arrowY_2012 - arrowSize_2012 * Math.sin(angle_2012 +
Math.PI / 6))
            };

            g2d.setColor(new Color(100, 100, 100));
            g2d.fillPolygon(xPoints_2012, yPoints_2012, 3);
        }

        private void drawLegend(Graphics2D g2d) {
            int legendX_2012 = 20;
            int legendY_2012 = getHeight() - 70;

            g2d.setColor(new Color(255, 255, 255, 200));
            g2d.fillRoundRect(legendX_2012 - 10, legendY_2012 - 10, 400, 80,
10, 10);
            g2d.setColor(Color.BLACK);
            g2d.setStroke(new BasicStroke(1));
            g2d.drawRoundRect(legendX_2012 - 10, legendY_2012 - 10, 400, 80,
10, 10);

            g2d.setFont(new Font("Segoe UI", Font.BOLD, 11));
```

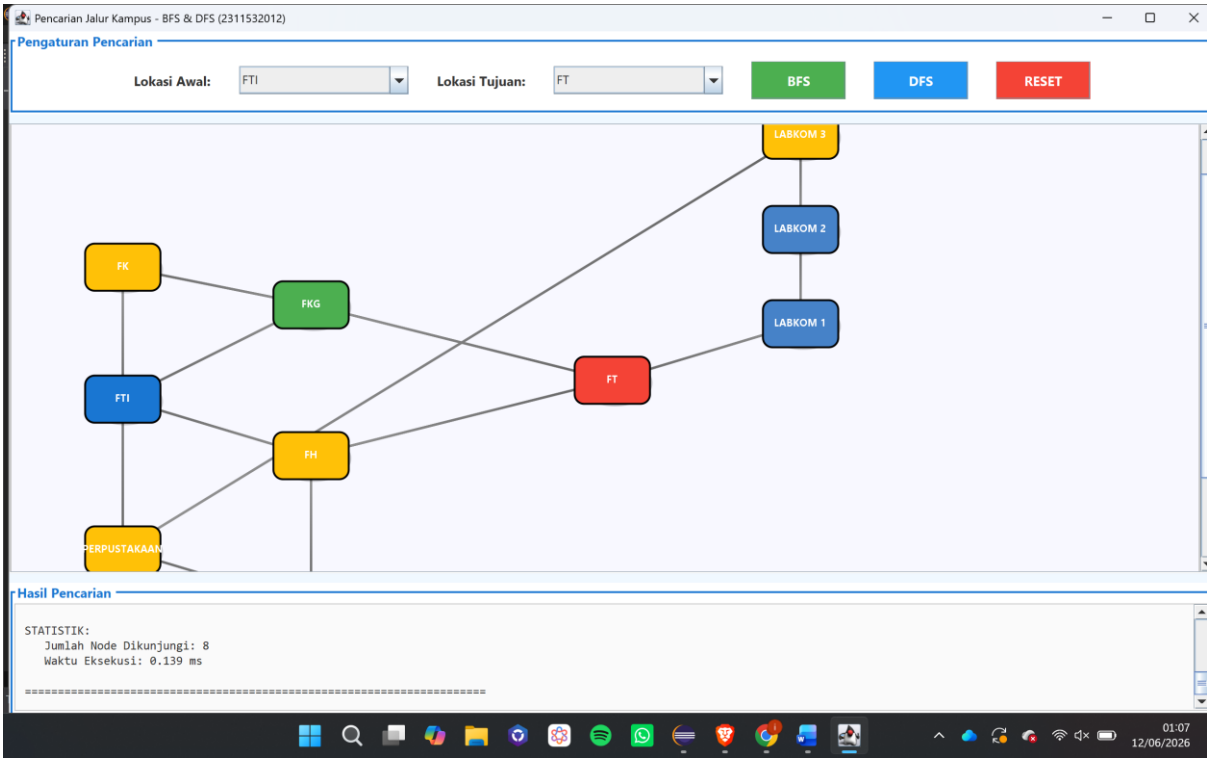
```
            g2d.drawString("Legenda:", legendX_2012, legendY_2012);

            g2d.setColor(new Color(25, 118, 210));
            g2d.fillRoundRect(legendX_2012 + 70, legendY_2012 - 8, 20, 15,
5, 5);
            g2d.setColor(Color.BLACK);
            g2d.drawString("Start", legendX_2012 + 95, legendY_2012 + 2);

            g2d.setColor(new Color(244, 67, 54));
            g2d.fillRoundRect(legendX_2012 + 145, legendY_2012 - 8, 20, 15,
5, 5);
            g2d.setColor(Color.BLACK);
            g2d.drawString("Goal", legendX_2012 + 170, legendY_2012 + 2);

            g2d.setColor(new Color(255, 193, 7));
            g2d.fillRoundRect(legendX_2012 + 215, legendY_2012 - 8, 20, 15,
5, 5);
            g2d.setColor(Color.BLACK);
            g2d.drawString("Dikunjungi", legendX_2012 + 240, legendY_2012 +
2);

            g2d.setColor(new Color(76, 175, 80));
            g2d.fillRoundRect(legendX_2012 + 215, legendY_2012 + 15, 20, 15,
5, 5);
            g2d.setColor(Color.BLACK);
            g2d.drawString("Jalur Hasil", legendX_2012 + 240, legendY_2012 +
25);

            g2d.setColor(new Color(70, 130, 200));
            g2d.fillRoundRect(legendX_2012 + 70, legendY_2012 + 15, 20, 15,
5, 5);
            g2d.setColor(Color.BLACK);
            g2d.drawString("Belum Dikunjungi", legendX_2012 + 95,
legendY_2012 + 25);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new PetaKampus_2311532012());
    }
}
```

2.3 Hasil

- **DFS**


Pencarian Jalur Kampus - BFS & DFS (2311532012)
Pengaturan Pencarian
Lokasi Awal:
FTI
Lokasi Tujuan:
FT
BFS
DFS
RESET
LABKOM 3
LABKOM 2
LABKOM 1
FK
FKG
FT
FTI
FH
PERPUSTAKAAN
Hasil Pencarian
STATISTIK:
Jumlah Node Dikunjungi: 10
Waktu Eksekusi: 0.105 ms
01:06
12/06/2026

- **BFS**


Pencarian Jalur Kampus - BFS & DFS (2311532012)
Pengaturan Pencarian
Lokasi Awal:
FTI
Lokasi Tujuan:
FT
BFS
DFS
RESET
LABKOM 3
LABKOM 2
LABKOM 1
FK
FKG
FT
FTI
FH
PERPUSTAKAAN
Hasil Pencarian
STATISTIK:
Jumlah Node Dikunjungi: 8
Waktu Eksekusi: 0.139 ms
01:07
12/06/2026